

Document intelligence

How a file becomes a record, and when it must stop
before it does.

Andre Naidoo

Technical assessment · September 2026

Code · <https://github.com/Swapmatics/team5pm-document-intelligence>

Superuser · <https://docintel.ziton.tech/superuser>

Finance · <https://docintel.ziton.tech/finance>

Projects · <https://docintel.ziton.tech/projects>

Accounts · <https://docintel.ziton.tech/accounts>

Confidential

CONTENTS**What this document is**

This is the technical assessment for team5pm, Johannesburg, September 2026. It answers how a file becomes one current record, and when the system must hold instead of guessing. A wrong total written as final is the failure. The later pages separate what is running from what is still a design, and they link the desk, the sheet, and the rows a reviewer can open.

THE ARGUMENT

The job. A wrong total, written as final, is the failure

The life of a row. Received, then a decision

Where the work sits. The desk, the queue, the book

What built this

Why it runs, not only reads

THE BRIEF

Part A. Foundations

A1. Webhook, and a manual or form trigger

A2. Waiting, and not waiting

A3. A 429 from the model

A4. Self-hosted n8n, against n8n Cloud

A5. Idempotent

Who may see it. Security and access

Part B. The model proposes. The checks decide.

Part B. One document, one current line

Part B. Every failure has a visible end

Part B. A contract is a set of pages

Part C. A person, a backfill, a quiet morning

A correction without opening n8n

Ten thousand old files

Healthy, without the execution log

WHAT YOU CAN OPEN

Proof. Production acceptance

Beyond the brief. Where to open the build

What you can press

What I would ask first

Appendix. What I can point at

References

THE JOB

A wrong total, written as final, is the failure

A run that stops and says why is a success. That is the policy. The rest of this document is how I keep it true when nobody is watching, and how I prove it on the desk.

The file can be a contract, an invoice, a brief, or a signed agreement. Someone drops it. The system has to say what it is, who it is with, what it is worth, and when it applies. If it cannot say that from the page, it holds the file. It does not fill the gap.

Five rules sit under every step.

1. A required value is kept only when it is on a page I can point at.
2. Every failure ends in a state a person can see. There is no path that continues with a hole.
3. The file can sit unattended. Received is not approved.
4. The thread and the row say what happened, and neither can be quietly overwritten.
5. Who the person is, and which rows they may see, are two different checks.

A document is done when the file is stored, the identity is settled, the read has finished, the required fields have passed, each of those fields has a page, the duplicate and revision checks have passed, the current-row write has committed, the downstream view has been updated or queued, and an audit line has been written. A green execution in n8n is not that list. n8n decides the next step. It does not own the rows or the files.

The desk proves the state changes that matter: a clean invoice, the same file twice, a revised total, a blank page, and a scan whose words are not a total. The rows are in the appendix. Slack, Drive, and the mailbox are running, and they post into the same intake. The rows I can point at were submitted from the desk. A real login is not in this build. The link is still the seat.

THE LIFE OF A ROW

Received, then a decision

The database row starts as received. The worker claims it as processing. From there it becomes accepted, held, or waiting. Accepted with nothing pointing at a newer row is current. Waiting is a yes or a no. Silence stays waiting.

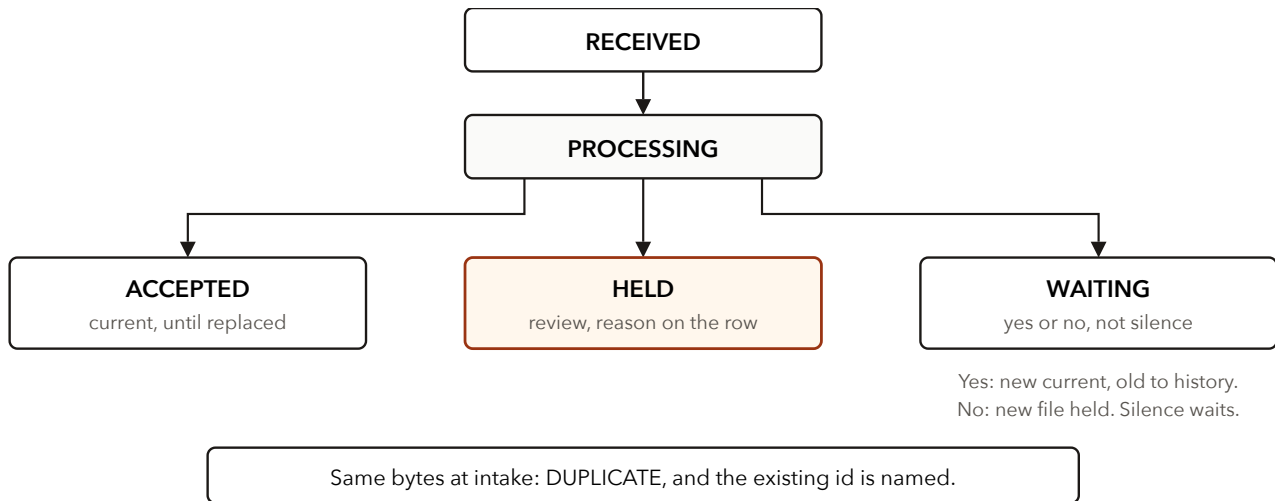


Figure 1. One row, one current record. A duplicate never becomes a second document.

The same bytes are recognised before a model call. The hash is unique in Postgres. If two uploads of those bytes arrive together, one insert wins and the other is told which document it already is. A different file with the same invoice number or contract reference does not overwrite. It waits. Yes replaces the current row and moves the old one to history in one transaction. No holds the new file. If that transaction fails, nothing has moved.

WHERE THE WORK SITS

The desk, the queue, the book

A person opens a page on this machine. They do not open n8n. The page knows who the link is for, asks what the file is, and shows the book: reading, current, held, history. Five cases sit on that page. Each one runs the real intake.

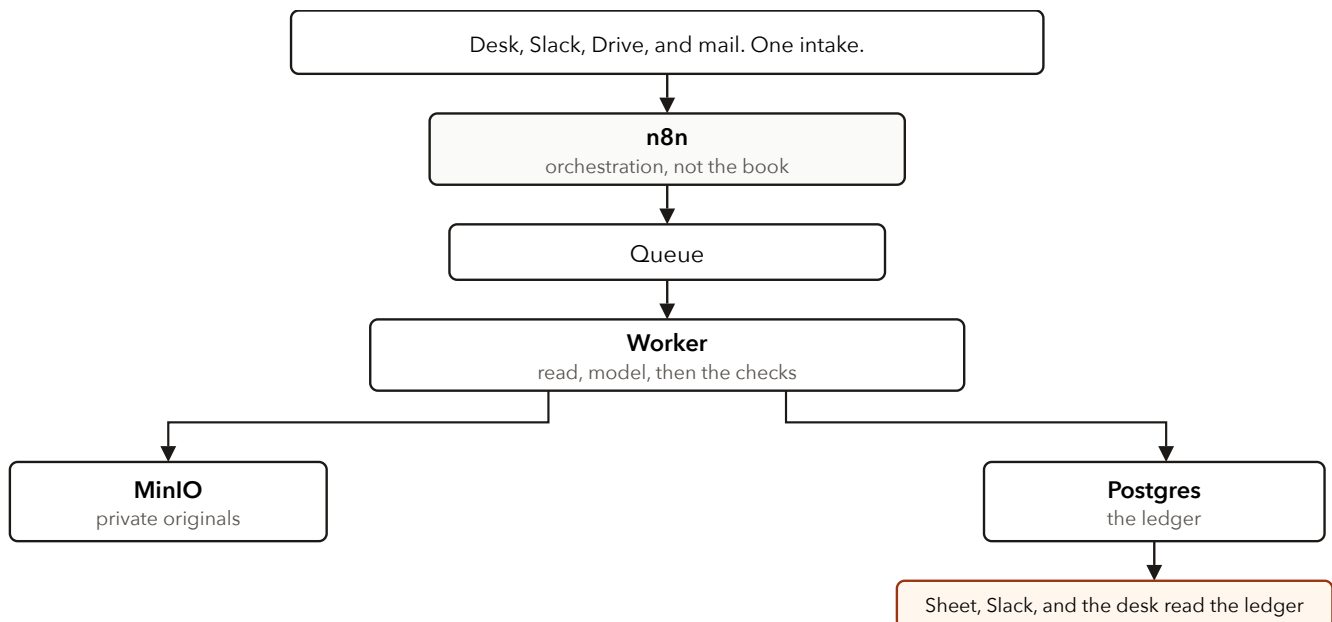


Figure 2. n8n starts the work. The file lives in the bucket. The row lives in Postgres.

On this machine the path is already that shape. Intake hashes the file. The same hash is named and stopped, with no model call. Two uploads of those bytes at the same moment take one lock. The second waits, then is told the existing id. The book has PDFs and one Word file. A PNG or a JPEG has

an image route. There is no photo in this book. A Word file is read as a Word file. A stated type the text disagrees with is held, and not relabelled. That check is in the judge. It is not one of the buttons. The original goes to a private bucket, a received row is inserted, the job is queued, and the person is told the file is not approved yet. The worker then claims the row, reads it, and writes accepted, held, or waiting. The browser is not holding the HTTP call open for the model.

ClamAV 1.5.4 scans the file before it is opened, and again when the worker asks the desk to read the pages. A flagged file is refused, and no row is written. If the scanner does not finish, the file is not opened. I have not put a test virus through the desk for this submission.

ON THIS BUILD

You can run it

- Desk, with the five cases in the appendix
- Slack, Drive, and the mailbox, into that same intake
- Hash lock, and one row in Postgres
- Private bucket, and ClamAV before open
- Queue, worker, received then processing
- OCR, including a scan whose words are not a total
- A file longer than ten pages, in groups of about ten, one call at a time
- An update flag. A new file is kept, and every door says the desk is not offline.
- Sheet copy, with a link to the file. Amounts stay on the sheet.
- Yes or no, and a correction on the desk
- Department views on the desk. Projects and accounts see no finance rows.

WRITTEN, NOT RUNNING

Still to do

- Google login, instead of the link
- A company-email rule on the live inbox
- A real Retry-After wait on a 429
- A second seat whose sheet hides amounts
- A cleared message when an alert ends

WHAT BUILT THIS

The model proposes. I decide.

Cursor wrote most of the code, inside the architecture above. It does not choose the architecture. ChatGPT and Claude were the reasoning partners: the failure taxonomy, the retry policy, and the column schema were argued through with them before a line was written. Neither approved a value or touched a row in Postgres. Same rule as the rest of this document: the model proposes, the person and the code decide.

WHY IT RUNS, NOT ONLY READS

I build to find the break

A written design can describe a rule. It cannot tell you where that rule breaks. Under a real file, a slow API, a second upload of the same bytes. I build to find the break before a reviewer does, not to argue I would have found it eventually. That is not extra effort for this exercise. It is how I harden a process: by running it, not by scoping it well and hoping the gaps stay theoretical.

PART A

Foundations

A1. Webhook, and a manual or form trigger

A webhook is a URL another system calls. The desk, Slack, Drive, and the mailbox already have the file, or a pointer to it. The workflow starts because that system posted it.

A manual trigger is the button inside n8n. I use it while I am building. It is not how finance submits a contract. A form trigger is a page n8n hosts. I would use it if I had no other page. I built the desk because the thread, the book, and the yes or no need to live on one screen. I do not want the person on an n8n URL.

A2. Waiting, and not waiting

Synchronous means the caller sits until the work is finished. Asynchronous means the caller is told the file was taken, and the work happens after that call has ended.

Intake on this build is the second kind. It stores the file, inserts received, queues the worker, and replies. The reply is that the file is not approved yet. The case buttons on the desk then watch the book until the row leaves reading. A 100-page file takes the same first reply. It must not hold a browser open. The reader groups that file after the reply. I have not put a 100-page file on the book.

A3. A 429 from the model

HTTP 429 means the service asked us to slow down. It is not a bad file. A 400 is a bad request. A 401 is a bad key. I do not retry those. Retrying them makes noise and can hide a broken configuration.

The model call on this build tries four times, three seconds apart. That is a start. The policy I want is: honour Retry-After when the service sends it, then wait longer each time with a little randomness, and stop at a retry budget. Past that budget the row is held and the thread says the model service asked us to wait. Fields stay blank. An empty total is not zero.

A retry is the same document id. It must not insert a second row, and it must not accept a second current record. The id is assigned at intake, before the model is called.

A4. Self-hosted n8n, against n8n Cloud

The contracts stay on a machine we run. Queue mode, Postgres, and the bucket sit next to n8n. Execution data can contain page text. I do not want that text on someone else's cloud as the price of not running a box. We are also not billed per execution while a worker reads a long file.

The cost of that choice is the box. Upgrades, backups, disk, the encryption key, and a worker that has quietly stopped are ours. Cloud would take that class of failure off the table. For this workload I still self-host, and I treat backups of Postgres and the bucket as part of the system.

A5. Idempotent

Doing the thing twice leaves the same result as doing it once. A double click, a retry after a timeout, a mailbox and a person both submitting the same PDF: one document, not two.

The key is the hash of the bytes. The column is unique. The lookup happens before the model. If two inserts race, one wins and the other receives the existing id. A worker that dies after the row is already accepted does not write it again. The update runs only while the row is received or processing. A second delivery of a finished document is skipped.

WHO MAY SEE IT

Security and access

Authentication says who the person is. Authorization says which documents and which fields they may see. On this desk, the link `?as=andre` stands in for the first of those. It is a named seat in a staff map, so the department filter and the thread can be shown. It is not a login. Production removes it. The person authenticates with Google, or with the Slack identity already stored against that person, and the book is checked before the original file or a sensitive field is returned.

The header secret on the webhook stops a stranger calling n8n directly. It does not prove who the person is. The browser never receives the database password, the model key, or the storage key. Those sit outside the workflow data and can be rotated on their own.

On the desk, a projects or accounts seat does not receive payment amounts. The page does not merely hide a column that the query already returned. Those two seats currently see an empty book, because every row is finance. A superuser sees the set. The Google Sheet is a separate copy, and that copy still includes currency and totals for anyone with the link. Hiding amounts on a second sheet is not built. Nobody gets a delete. A wrong row is superseded and stays in history. The thread is append-only. Documents and rows are kept for a retention period I would write down with the company before go-live. I have not invented that period here.

PART B

The model proposes. The checks decide.

I do not treat the model's reply as the record. It returns one JSON object. Temperature is zero. The instruction says to leave a field empty when it is not on the page, and to list a clash instead of picking a winner. Ordinary code then decides whether those values are allowed to become truth.

TASK	WHO DOES IT
File hash	Code
Duplicate detection	Postgres, unique hash
Who the person is	Identity provider. On this desk, the link is a stand-in.
Which rows they may see	Code and the book
OCR	Apple Vision, on a page with no text. An empty page is held and named.
Reading the fields	The model
Naming a conflict	The model lists it. Code refuses to pick.
Date, amount, required fields	Code
Replacing the current row	One Postgres transaction
Retry	The workflow, inside the same document id
The queue	Redis and the worker
A yes or a no	The person
The audit line	Postgres

This build calls one route: OpenRouter, model id google/gemini-2.5-flash, temperature zero. The row stores that route. The checks do not depend on which model filled the JSON. Google’s current guidance is that new projects should prefer a newer model, and that access to 2.5 can be limited. Swapping the route is a configuration change. It is not a new book, and it is not a reason to accept a total the page does not support. The workflow also has an image route for a PNG or a JPEG. No photo is in this book. On an invoice the counterparty is the supplier, not the customer. If the person marked the type and the text looks like a different type, I hold the file and I say so. I do not relabel it. The type decides which fields are required.

For a text PDF, a required value has to appear on a page. If it does not, the field is blanked and the document is held. The row records the sentence: the total, the reference, the date, the party, and the currency, each with its page. A total with no page is not a total. A page with no text is rendered and read with Apple Vision. If it is still empty, the document is held and the page is named. If the scan comes back with words, and those words are not a total, the total stays blank. That page is not sent on as a guess. A Word file cites the section, not an invented page number.

The system does not assume the model output is correct. Unsupported or conflicting values are rejected. The model output is a candidate extraction, not the book.

PART B

One document, one current line

Postgres is the book. The Google Sheet is a copy of those columns, split into Current, Held, Reading, and History, so a revised total is not two live invoices. Each row links to a viewable copy of the file. If the sheet is down, the book is still the book, and the sync retries. The sheet is not where the workflow writes first.

COLUMN	WHAT IT IS
document_id	Stable id, from the hash and a time suffix. Assigned at intake.
type, department	The type they stated. The department comes from the staff map.
parties, counterparty	Parties are required on a contract. On an invoice, the counterparty is the supplier.
dates	Document, effective, expiry. A real calendar day, or blank. Effective date is required on a contract.
currency, total	Digits and a decimal. No symbol in the amount. Both required on an invoice.
reference	Invoice number or contract reference. This is how a revision is recognised.
why it exists	Required on a brief.
submitted_by, accepted_at	Who sent it, and when it became current.
original	A storage key. The desk streams it only after the row says this person may see it.

The ledger also keeps the hash, the storage key, page notes, validation notes, which model ran, who reviewed it, and the id of the row this one replaces. An invoice needs the supplier, the invoice number, the currency, the total, and the invoice date. A contract needs the parties and the effective date. A brief needs the title and why it exists. Anything marked other needs a title.

PART B

Every failure has a visible end

I group failures so a reviewer can see there is no silent branch.

CLASS	EXAMPLES	WHERE IT ENDS
Input	Unsupported file, corrupt file, malware, missing type or department	Refused before a row is written. A scan that flags the file, or does not finish, does not leave a received row.
Extraction	OCR failure, model timeout, junk JSON, provider down	Held. Fields blank. The thread says the read did not return a usable result.
Validation	Missing field, bad date, bad amount, two values	Held. The bad field stays blank. Both conflicting values stay in the note. Nothing is picked.
Business rule	Duplicate, revision, more than one current match	Duplicate names the old id. A revision waits. An ambiguous match is held and the ids are listed.
Infrastructure	Database down, bucket down, worker down, queue growing	The job stays queued or fails in the open. It does not become current.

Every class has a visible terminal state. There is no path where the workflow continues with incomplete data.

The next table is the policy I want. It is not a claim that every row of it is already in the node.

ERROR	RETRY	THEN
429	Yes. Retry-After, then a longer wait, inside a budget.	Hold, and say the service asked us to wait.
408, timeout, 5xx	Yes, limited.	Hold if the budget is spent.
400	No	Hold, and inspect the request.
401, 403	No	Alert. This is a key or a permission, not a bad PDF.
Junk JSON	Once or twice, then stop.	Hold. Never accept.
OCR empty	Limited, then one picture of that page.	Hold and name the page if it is still empty.
Database unavailable	Yes	Leave the job on the queue.

This build retries the model call four times, three seconds apart. The longer wait, the randomness, and the Retry-After header are the policy above. They are not in the node yet. What is in the node is the rule that a failed call does not become an accepted row.

If the worker dies halfway through a document, the row is still processing. Queue mode is how this build runs, so a job that is not finished can be delivered again under the same document id. The write runs only while the row is received or processing. A crash cannot create a second current record. A crash after the row is already accepted is a skip, not a second approval. I have not killed a worker in the middle of a document for this submission.

PART B

A contract is a set of pages

A one-page invoice is one text block, one model call, one decision. A 100-page contract is a pile of pages. Some have text, some are scans, some are blank on purpose. The person already has the reply that the file is received and not approved. The job id is the document id. Sending the file again hits the same hash.

The reader walks the pages. Text stays text. A page with no text goes to OCR. If OCR is still empty, the page is named and the file is held. That page is not sent on. A long file is then grouped, about ten at a time. Each group is its own call, and each candidate keeps its provenance: which chunk, which pages, which field, which value, which source page, and a short excerpt. The merge sees that chunk 4 says 18,000, from page 37. If two chunks disagree, both values are kept and the file is held.

A file of ten pages or fewer is one call for each page that has text. Past that, the reader splits the pages into groups of about ten. Each group is one call. The call is told the range, pages 21-30 of a longer document, and it may return a field only when those pages support it. It says when the range reads as a continuation. Temperature stays zero. A blank field is left blank. That call uses the same four tries, three seconds apart.

The groups run one at a time, with a second and a half between calls. One call is in flight. The limit around it is two or three, so a hundred-page file does not send ten calls at once.

The merge is the rule this build already uses. No group offered a value, and the field stays blank. One value is the candidate, with the page it came from. The same value from two groups is kept. Different values are both kept, nothing is picked, and the file is held.

A one-page invoice is still one call. The blank-page rule is unchanged. A contract whose second page has no text is held, and the note names page 2. I checked the grouping and the merge against made-up chunk results. I have not put a 100-page file on the book.

Cost is kept down by doing the cheap work first. The hash and the duplicate check are code. OCR runs only where the page has no text. The model runs only where a person would otherwise have to interpret the page. The price list is secondary, and it moves. Prices checked on 22 September 2026. Flash is 0.30 US dollars per million input tokens and 2.50 per million output tokens. Thinking tokens are billed as output. Flash-Lite is 0.10 and 0.40.

JOB	ROUGH SIZE	ROUGH COST, THINKING OFF
One-page invoice	~600 in, ~400 out	under 0.2 US cent
100-page contract, one call	~60,000 in, ~800 out	about 2 US cents
Same contract, ten chunks	ten smaller calls	still under 10 US cents
10,000 text documents, ~8 pages	~48 million in	on the order of 30 US dollars

Those counts are estimates from a typical page, not a measured bill from this project.

I split a long contract so the merge can refuse a clash, and so the browser is not the thing waiting. A few cents is not the reason. Thinking left on can also turn a cheap call into a much larger output bill, for a job that is a form. I would turn thinking off for the field pass, and I would measure that against the fixtures before I trusted it. Flash-Lite is the trial model for the cheap questions: is this page empty, does the stated type look wrong. I would not move the money fields to Lite until the same fixtures come back with the same holds and the same totals.

PART C

A person, a backfill, a quiet morning

A correction without opening n8n

A waiting row asks yes or no on the desk. A held row can be corrected there too. Each changed value needs the page it came from. Saving the review checks the fields again and does not read the file or call the model. It becomes current only when the required fields are valid and every named blank page has been acknowledged.

Ten thousand old files

An archive does not go through the afternoon door. It is a second queue. It is admitted in small batches only while no live file is still being read. A hash already on the book is skipped. A run of failed reads, or a held majority in the last archive batch, pauses it. The door stays idle until a folder is set. A waiting list of 10,000 files was shown, then rolled back. It did not spend a model call.

Healthy, without the execution log

The desk shows whether n8n, the book, Redis, the bucket, and the sheet answer. It also shows today’s model spend, and how many pages could not be read. On 23 September 2026 that line reported all five up, two pages that could not be read, and about 0.002 US dollars of model spend. The superuser is told when a file has been waiting to be read, when a read failed before a result came back, and when a

held file has been waiting on a person. The message is sent once. When the condition clears, it is not sent again. There is no separate cleared message. A person who sent a file by email is told if it was not stored, or if it did not finish as an approved record. Silence is not a success.

PROOF

Production acceptance

The primary safety measure is the false acceptance rate. A document counts as a false acceptance if it is marked current and a required value is wrong, or is not supported by the page. A false hold is a good file stopped for a person. A false acceptance is the more serious of the two. The system is biased toward holding an uncertain document.

I would not use the model's own sense of certainty as that proof. Each fixture has an expected value for every required field, including the page. Model output is compared with that expected result before a model, a prompt, or a version is promoted. A change that improves one field and degrades another is not promoted on its own. The measures I would keep are field accuracy, required-field recall, false acceptance rate, false hold rate, OCR failure rate, duplicate rate, processing time, and cost per document. I do not have a scored set large enough to publish a rate from, and I will not invent one.

In the table, Shown means a row in the appendix. In the checks means the code decides it, and I do not have a separate row for it.

TEST	EXPECTED RESULT	THIS BUILD
Valid invoice	Accepted, and current	Shown. INV-1001, 12500.00 ZAR, page 1.
Same file twice	One document, one model path	Shown. The existing id is named.
Two identical uploads at once	One insert wins. The other names it.	Shown. Two callers, one row.
Revised invoice	Waiting. The current row is unchanged.	Shown. 18000.00 waits on 12500.00.
Revision approved	New row current. Old row to history, in one transaction.	Shown. INV-2044. 9400.00 is superseded. 11200.00 is current.
Revision rejected	New file held. Previous row stays current.	The no button is on the desk. INV-1001 at 18000.00 is still waiting. I did not press no.
Missing required field	Held. The field stays blank.	Shown on the scan. The total stayed blank.
Conflicting totals or dates	Held. Both values kept.	In the checks. Nothing is picked. That scan is not its own button.
Invalid date or amount	Held.	In the checks.
Scan whose words are not a total	Held. The page was read. The total stays blank.	Shown. Page 1 of the scan. The words were not a total, and nothing was guessed.
Blank or unreadable page	Held, and the page is named.	Shown. Page 2 of the contract fixture.
Malformed model JSON	Held. Never accepted.	In the checks, when the reply is junk.
Model timeout or 429	Retry inside the policy, then hold.	Four tries, three seconds apart.
Worker unavailable	The queue grows, and an alert fires.	The alert is in the desk. A dead worker was not staged for this submission.
100-page document	Asynchronous. Groups of about ten pages. A clash is held.	The reply is already asynchronous. The reader groups the pages. A 100-page file is not on the book.
Unauthorized person	No document, and no restricted field.	Department filter. The link is not a login.
Sheet unavailable	The book remains Postgres. Sync retries.	The sheet is a copy of the book. I have not switched it off for this submission.

The system is not production-ready until these cases pass against the same build that is demonstrated.

The regression set

Functional: a clean invoice, a contract, a brief. Adversarial: a blank page, a corrupt PDF, conflicting totals, an impossible date, junk JSON. Concurrency: two identical uploads at once, two revisions at once, a worker restart during processing. Dependencies: a 429, a timeout, a dead model, a dead database, a dead bucket. Scale: one 100-page document, a 10,000-file backfill, and live documents at the same time. Security: a person outside the department, a session that has expired. The five buttons cover the first slice of that list. They are the cases I will not weaken to make a demo look finished.

BEYOND THE BRIEF

Why these are running, and where to open them

The brief asked for a design. I also built the path, because a hold that cannot be opened is only a sentence. Each extra is a door or a copy a person can look at without opening n8n or the database. The book stays Postgres. These are how you see it.

EXTRA	WHY IT IS HERE	OPEN IT
Web desk, four seats	The thread, the book, and yes or no are one page. Four seats make the access rule visible. Projects and accounts do not receive amounts.	Superuser , Finance , Projects , Accounts . No password.
Google Sheet	The brief asks for one structured record per document. The sheet is that copy, in four tabs: Current, Held, Reading, History. A file link sits on the row. Amounts are on this copy. The workflow does not write here first.	Open the sheet . The same link is on the superuser status line.
Mailbox	A file has to be able to arrive when nobody is at the desk. The demo inbox accepts any sender so the path can be tried. Production refuses any From address that is not a company email. That filter is written, and it is not enforced on this inbox.	Send a PDF to team5pmdocintel@gmail.com . Then refresh Finance and the sheet. The row is the proof.
Slack	The production door is a chat. The person is the chat, not a name they type. Department comes from the staff map.	The bot replies in a direct message. A reviewer verifies that door on the desk and on the sheet: the same row appears in both.
Drive	A file already in Drive should not be downloaded and uploaded again by hand. The folder is the same intake.	DocIntel intake . A new file lands on the desk and on the sheet.
Five case buttons	The holds have to be pressed. Clean invoice, the same file, a revised total, a blank page, and a scan whose words are not a total.	On the superuser seat. The result is in this book and on the sheet. The appendix names the rows.
Status line	Health without opening the execution log. n8n, the book, Redis, the bucket, the sheet, today's model spend, and pages that could not be read.	The line at the top of each seat.

The code is on the cover. ClamAV, the queue, and the judge are in that repository. The rows and the links above are how you see them running.

WHAT YOU CAN PRESS

The desk, and what it is not

These four links open the desk. There is no password. Each one is that seat's book. A person outside finance does not receive currency or totals. The link is the seat. It is not a login.

- Superuser · <https://docintel.ziton.tech/superuser>
- Finance · <https://docintel.ziton.tech/finance>
- Projects · <https://docintel.ziton.tech/projects>
- Accounts · <https://docintel.ziton.tech/accounts>

The status line shows whether n8n, the book, Redis, the bucket, and the sheet answer. The buttons are Clean invoice, Same file again, Revised invoice, Blank page, and Wrong words.

A clean invoice comes back accepted, with the supplier, the invoice number, the total, and the page each value was found on. The same file comes back as the same document. A revised total comes back waiting, with the old total and the new one both named, and the current row left alone until a person answers. A contract with an empty second page comes back held, and the note names page 2. The parties and the effective date can still be cited. The file is not accepted.

The brief's file formats are a native-text PDF, a scanned image, and a Word document. The book has the PDF cases and one Word invoice, INV-3001, cited from section 1. A picture embedded in a Word file is read with the same OCR as a scanned page. It is not converted into a PDF first. A PNG or a JPEG uses the image route. There is no photo in this book.

The demo mailbox is team5pmdocintel@gmail.com. For the demo it accepts a file from any sender, so the path can be tested. That is not the live rule. Once this is in production, the inbox accepts a file only when the From address is a company email address. Mail from any other domain is refused. It is not stored, and it is not sent to the model.

Production still replaces the link with a real login, and honours a real Retry-After. The holds and the hash are rows I can name. ClamAV runs before a file is opened. Slack, Drive, and the mailbox are running doors. Before a restart I set an update flag. While that flag is up, a new file on any door is kept and the person is told. The sentence is not optional. It says an update is in progress, the file is kept, and the desk is not offline. I would rather show a hold I can explain than a login I have not built.

WHAT I WOULD ASK FIRST

Four answers that would change the build, not the config

A design that stops before it knows should also say what it does not know yet. Four questions sit under this one. I have put them in the order they would change the most.

Can page content leave the building

This build sends every page to OpenRouter, then Gemini. If a contract carries a confidentiality clause, or if legal has a rule against sending unredacted text to a third party, the model call is a different reader. A self-hosted open-weight model, or a signed no-retention agreement with the provider. That is a different cost line, a different accuracy ceiling, and possibly a GPU the company does not yet have. I would ask this first. It is the one answer that is a change of system, rather than a setting.

How many documents, and how many at once

A stream can mean twenty documents a day or two thousand. This build is one worker on one queue. Real volume changes the worker count, how many calls the code may have in flight against the provider's own limit, and whether Postgres alone still holds the write load. The checks stay. The number of machines changes.

What the record feeds next

The brief allows a sheet or a database, and this build is a sheet. If finance's next step is Xero, or legal's is a contract system, the sheet stays a staging copy. An output adapter, with its own sign-in and its own rule against writing the same row twice, joins the record.

What the company already runs

This build's doors are Slack, Drive, and Gmail. A Microsoft shop makes those doors Teams, SharePoint, and Outlook. The login I have not built becomes Entra ID, where this one would have been Google. If a signed agreement has to show it was not altered, keeping the row is the start. A periodic signed manifest of the ledger would sit above it.

I do not have these answers yet. That is why they are written here. A held field is a better answer than an invented one.

APPENDIX

What I can point at

These are rows in the book on 23 September 2026. Shown in the proof table means one of these.

ROW	WHAT IT SHOWS
DOC-52fa12fd-mud4ox8g	Accepted. INV-1001, 12500.00 ZAR, page 1.
DOC-4b390e46-mud4p6vv	Waiting. INV-1001, 18000.00. The current 12500.00 row is unchanged. I have not pressed yes or no.
DOC-7bf0eb5f-mud2qdne	History. INV-2044, 9400.00, superseded.
DOC-f4404749-mud2qs7	Current. INV-2044, 11200.00, after a yes. One transaction.
DOC-bb9b09a0-mud4vupx	Held. CTR-55. Page 2 has no text.
DOC-677f720a-mud2tx0z	Accepted only after a person acknowledged the blank page. CTR-88. Reviewed by Andre.
DOC-5321f129-mudzyooh	Held. The scan was read. The words were not a total. The total is blank. Page 1 is named for the fields that were kept.
DOC-6fa92327-mudv9y4t	Accepted Word invoice. INV-3001, 1500.00, section 1.
DOC-ff2b8c08-mudr7oz5	Accepted OCR contract. CTR-99. The required fields are cited from page 1. A contract does not need a total.

Two earlier scans of the same wrong-words invoice are also held, with a blank total. The row that carries the sentence above is DOC-5321f129-mudzyooh.

On that morning the status line reported n8n, the book, Redis, the bucket, and the sheet up. Projects and accounts each returned an empty book. Finance returned the finance rows, including amounts.

Not a row, and not claimed as shown: a PNG or a JPEG, a pressed no, a killed worker, a switched-off sheet, a staged 429, a test virus, a company-email refusal, a login, a 100-page file, and a cleared alert. Those are either in the checks, running as doors, or still to build. The lists above say which.

REFERENCES

n8n, Webhook node. [docs.n8n.io](#). The intake, confirm, book, and thread triggers.

n8n, Form trigger and Manual trigger. [Form](#), [Manual](#). The distinction in A1.

n8n, Queue mode. docs.n8n.io/hosting/scaling/queue-mode. Redis and a worker. This build runs that way, on port 5680, separate from any other n8n on the machine.

n8n, HTTP Request node, retry on fail. docs.n8n.io. Four tries, three seconds apart, on the model call.

Google, Gemini API pricing, checked 22 September 2026. ai.google.dev/gemini-api/docs/pricing. Flash 0.30 / 2.50 per million tokens. Flash-Lite 0.10 / 0.40. Output price includes thinking tokens.

Google, Gemini 2.5 Flash model page. ai.google.dev/gemini-api/docs/models/gemini-2.5-flash. New projects are pointed at a newer model, and access to 2.5 can be limited. This build keeps the model id on the row so the route can change without a new book.

OpenRouter, Gemini 2.5 Flash, same rates on that date. openrouter.ai/google/gemini-2.5-flash. The route this build calls today.

Token counts in the cost table are estimates, not a measured bill from this project.